An Introduction to Machine Learning for Physicists

A Theoretical and Practical Guide

David Hutasoit

September 2026

Copyright

An Introduction to Machine Learning for Physicists

A Theoretical and Practical Guide

David Hutasoit

Digital Open Access & Non-Commercial License

The digital version of this text and its accompanying Python notebooks are made freely available for educational, academic, and non-commercial use. You are permitted to:

- **Share:** Copy and redistribute the material in any medium or format.
- **Adapt:** Remix, transform, and build upon the material for non-commercial purposes, provided you give appropriate credit to the author.

Commercial and Print Restrictions

While non-commercial sharing is encouraged, all commercial rights are strictly reserved. No part of this publication may be reproduced, stored, distributed, or sold in any physical printed format (including self-publishing or through third-party publishing houses) or utilized for any commercial purposes without prior written permission from the author.

Feedback and Errata

If you find typos, errors, or have suggestions to improve the text, please submit an issue or contribution through the official project repository.

First Edition, 2026

https://github.com/davidteaching/intro-ml-physics

Preface

Machine Learning (ML) is one of the most exciting and dynamic areas of modern research and application [1]. Over the past decade, it has evolved from a specialised subfield of computer science into a transformative toolkit that permeates virtually every branch of science and technology. For physicists in particular, ML offers not just new computational tools, but a new way of thinking about data, models, and the very nature of scientific inference.

Why This Book?

The purpose of this book is to provide an introduction to the core concepts and tools of machine learning in a manner easily understood and intuitive to physicists [1]. While there exist many wonderful ML textbooks [4, 5, 3], they are often lengthy and use specialised language that can be unfamiliar to physicists. This book builds upon the considerable knowledge most physicists already possess in statistical physics, linear algebra, and dynamical systems in order to introduce many of the major ideas and techniques used in modern ML [1].

As Mehta and colleagues observe, "the availability of big datasets is a hallmark of modern science, including physics, where data analysis has become an important component of diverse areas, such as experimental particle physics, observational astronomy and cosmology, condensed matter physics, biophysics, and quantum computing" [1]. Moreover, ML and data science are playing increasingly important roles in many aspects of modern technology, ranging from biotechnology to the engineering of self-driving cars and smart devices [3]. Therefore, having a thorough grasp of the concepts and tools used in ML is an important skill that is increasingly relevant in the physical sciences [1].

A Physicist's Perspective on Machine Learning

What distinguishes a physicist's approach to ML from that of a computer scientist? The answer lies in our training. Physicists are accustomed to thinking in terms of symmetries, conservation laws, energy landscapes, and emergent phenomena. We are comfortable with coarse-graining, mean-field theories, and the renormalisation group. We understand that a model is not the reality—it is an approximation, valid in some regime, that captures the essential degrees of freedom.

This perspective is remarkably well-suited to modern ML. As Mehta et al. emphasise throughout their review, there are "many natural connections between ML and statistical physics" [1]. The bias-variance tradeoff is a statement about model complexity and generalisation that mirrors the tradeoff between flexibility and stability in physical systems. Regularisation is the imposition of a prior—a confining potential that prevents overfitting. Gradient descent is the motion of a particle in a potential energy landscape, subject to dissipative forces. Neural networks are spin systems with learned couplings. Generative models are variational free energy minimisation.

The Bias-Variance Tradeoff

One of the foundational concepts in ML is the bias-variance tradeoff, which Mehta et al. place at the very beginning of their review [1]. A model with high bias makes strong assumptions about the data and is inflexible—it may underfit, failing to capture the true structure. A model with low bias is flexible but has high variance—it may overfit, capturing noise rather than signal. The optimal model balances these two sources of error.

For physicists, this is a familiar tension. A simple model (e.g., a harmonic oscillator) is analytically tractable and generalises well, but may fail to capture anharmonic effects. A complex model (e.g., a full molecular dynamics simulation) can reproduce the data perfectly, but is sensitive to initial conditions and parameters. The art of modelling—whether in physics or ML—is finding the right level of description.

Energy-Based Models and Statistical Physics

A major theme of this book, following Mehta et al., is the deep connection between energy-based models in ML and statistical physics [1]. Many ML models can be understood as Boltzmann distributions over some set of degrees of freedom:

$$p(\mathbf{x}) = \frac{1}{Z} e^{-E(\mathbf{x})}, \tag{1}$$

where $E(\mathbf{x})$ is an energy function and Z is the partition function. This formulation unifies seemingly disparate models: maximum entropy models, Restricted Boltzmann Machines, Hopfield networks, and even deep generative models can all be viewed through this lens. The goal of learning becomes the determination of an energy function such that the resulting distribution matches the data—precisely the inverse problem of statistical mechanics.

What This Book Covers

This book is structured to take the reader on a journey from foundational concepts to cutting-edge applications. It seeks to find a middle ground between a short overview and a full-length textbook [1].

Chapter 1 establishes the conceptual bridge between physics and ML, covering entropy, information theory, the bias-variance tradeoff, and the interpretation of loss functions as energy landscapes.

Chapter 2 explores supervised learning through the lens of energy landscapes and gradient descent, covering linear and logistic regression, regularisation, and support vector machines.

Chapter 3 turns to unsupervised learning and dimensionality reduction, presenting PCA, K-means clustering, and topological data analysis as tools for discovering hidden structure.

Chapter 4 develops the profound connection between neural networks and statistical mechanics—from the perceptron as an Ising spin, to Hopfield networks as spin glasses, to Boltzmann machines and their thermal sampling.

Chapter 5 covers modern deep learning architectures, including multi-layer perceptrons, backpropagation, convolutional neural networks, and attention mechanisms, all interpreted through physical principles.

Chapter 6 presents generative models—Variational Autoencoders and Diffusion Models—through the frameworks of variational free energy and Langevin dynamics.

Chapter 7 takes a deep dive into probabilistic reasoning, covering Bayesian inference, Gaussian processes, and variational inference—essential tools for quantifying uncertainty and incorporating prior knowledge.

Chapter 8 explores reinforcement learning and its deep connections to optimal control and the Hamilton–Jacobi–Bellman equation.

Chapter 9 introduces quantum machine learning—the intersection of ML with quantum computing—covering parameterised quantum circuits, quantum kernels, and variational quantum classifiers.

Chapter 10 presents case studies from particle physics, astrophysics, condensed matter physics, and quantum information, demonstrating ML in action.

Chapter 11 concludes with a discussion of challenges and future directions, including interpretability, data scarcity, physics-informed learning, and hardware limitations.

The Practical Spirit

Following the example of Mehta et al., this book emphasises hands-on, practical engagement with ML [1]. Each chapter includes Python code examples that implement the key algorithms using NumPy, scikit-learn, PyTorch, and Qiskit. The code is designed to be run and modified, encouraging experimentation and deeper understanding.

As Mehta et al. note, a notable aspect of their review is "the use of Python Jupyter notebooks to introduce modern ML/statistical packages to readers using physics-inspired datasets (the Ising Model and Monte-Carlo simulations of supersymmetric decays of proton-proton collisions)" [1]. We adopt a similar philosophy here, using physics-inspired datasets—the Ising model, gravitational wave signals, particle collision events—to illustrate ML concepts in a familiar context.

Who This Book Is For

This book is intended for physics students and researchers who want to understand ML from first principles and apply it to their own problems. The prerequisites are a working knowledge of linear algebra, calculus, and basic probability theory—the standard toolkit of any physicist. No prior experience with ML or advanced programming is assumed.

Whether you are an experimentalist looking to analyse your data more effectively, a theorist interested in understanding the statistical mechanics of learning, or a student curious about this exciting field, we hope this book will serve as a valuable companion on your journey.

Acknowledgements

This book has been inspired by the remarkable review article by Pankaj Mehta, Marin Bukov, Ching-Hao Wang, Alexandre G.R. Day, Clint Richardson, Charles K. Fisher, and David J. Schwab: *A high-bias, low-variance introduction to Machine Learning for physicists*, published in *Physics Reports* in 2019 [1]. Their work has set a high standard for bridging the gap between physics and ML, and we are deeply grateful for their contribution to the field. We encourage all readers to consult the original review for a more comprehensive treatment.

We also acknowledge the many physicists and computer scientists whose work has shaped the field, and the open-source community that has made ML accessible to all.

David Hutasoit
September 9, 2026

Contents

Chapter 1

The Intersection of Physics and Learning

Machine learning (ML) is often presented as a collection of disparate algorithms—regression, classification, clustering, neural networks—each with its own heuristic justification. For a physicist, however, a unifying framework emerges naturally when we view learning through the lens of statistical mechanics, information theory, and dynamical systems. This chapter lays the conceptual groundwork for the rest of the book by translating core ML ideas into the physical language we know best: entropy, free energy, phase transitions, and dissipative dynamics.

1.1 Information, Entropy, and Thermodynamics

At the heart of both learning and physics lies the concept of *uncertainty*. In thermodynamics, entropy quantifies the number of microscopic configurations consistent with a macroscopic state. In information theory, entropy measures the average surprise or information content of a random variable. The deep connection between these two views—forged by Shannon, Boltzmann, and Jaynes—is the bedrock of a physicist's approach to ML.

1.1.1 Shannon Entropy

For a discrete random variable X taking values in $\mathcal{X}$ with probability mass function $p(x) = \Pr(X = x)$, the Shannon entropy is defined as

$$H(X) = -\sum_{x \in \mathcal{X}} p(x) \log p(x), \tag{1.1}$$

with the convention $0 \log 0 = 0$. This quantity satisfies three key properties:

1. *Non-negativity:* $H(X) \geq 0$, with equality iff X is deterministic.
2. *Maximum at uniformity:* For a finite alphabet of size $|\mathcal{X}|$, $H(X) \leq \log |\mathcal{X}|$, achieved when $p(x) = 1/|\mathcal{X}|$.
3. *Additivity for independent variables:* If X and Y are independent, $H(X, Y) = H(X) + H(Y)$.

In supervised learning, we often work with the conditional entropy $H(Y|X)$, which measures the remaining uncertainty in the target Y given the input X. The reduction in uncertainty—the mutual information $I(X; Y) = H(Y) - H(Y|X)$—is precisely what a good predictive model tries to maximise.

1.1.2 Gibbs Entropy and Statistical Mechanics

In statistical mechanics, the Gibbs entropy for a canonical ensemble is

$$S = -k_B \sum_i p_i \ln p_i, \tag{1.2}$$

where $p_i = e^{-E_i/(k_B T)}/Z$ is the Boltzmann distribution. Up to the factor k_B, this is structurally identical to Shannon entropy. The maximum entropy principle—choose the distribution that maximises entropy subject to known constraints—leads naturally to the Boltzmann factor.

Jaynes showed that this principle is not merely an analogy: statistical mechanics is a form of *inference* given incomplete information. From the ML perspective, when we regularise a model, we are effectively imposing a prior distribution, which can be interpreted as adding an entropy term to the loss function. This connection will reappear when we discuss variational autoencoders (Chapter 6) and Bayesian methods.

1.1.3 Cross-Entropy and the Kullback–Leibler Divergence

In classification tasks, we minimise the cross-entropy loss:

$$L = -\sum_i y_i \log \hat{y}_i, \tag{1.3}$$

where y_i is the true label (one-hot encoded) and $\hat{y}_i$ is the predicted probability. This is equivalent to minimising the Kullback–Leibler (KL) divergence between the true distribution p and the model distribution q:

$$D_{\mathrm{KL}}(p \parallel q) = \sum_i p_i \log \frac{p_i}{q_i} = H(p, q) - H(p). \tag{1.4}$$

Since $H(p)$ is fixed for the training data, minimising cross-entropy is the same as minimising the KL divergence – a measure of how much information is lost when using q to approximate p. In physics, the KL divergence appears as the relative entropy, measuring the irreversibility of thermodynamic processes.

1.2 Machine Learning as a Statistical Field Theory

A dataset can be viewed as a finite sample drawn from an unknown probability distribution $\mathcal{D}$ over an input space $\mathcal{X}$ and (for supervised problems) an output space $\mathcal{Y}$. Learning is the process of constructing a model distribution $p_\theta(y|x)$ that approximates the true conditional distribution. This is remarkably similar to constructing an effective field theory: we start from a high-dimensional, complex system (the data-generating process), and we seek a simplified description (the model) that captures the relevant degrees of freedom.

1.2.1 Datasets as Ensembles

Consider a dataset $\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)\}$. In a physical framework, this dataset represents a finite set of observations sampled from an infinitely large "ensemble" of all possible realities governed by an unknown joint probability distribution $P(X, Y)$.

The goal of supervised learning is to approximate the conditional distribution $P(Y|X)$. When we define a model (like a neural network) with a set of parameters $\boldsymbol{w}$, we are defining a parameterized family of distributions $Q_{\boldsymbol{w}}(Y|X)$.

1.2.2 Learning as Reaching Thermal Equilibrium

The process of "learning" involves updating the parameters $\boldsymbol{w}$ to make the model's distribution $Q_{\boldsymbol{w}}$ match the true data distribution P. We measure the "distance" between these distributions using the Kullback-Leibler (KL) divergence.

In the language of physics, the loss function we minimize during training acts as an effective Hamiltonian (energy landscape). The optimization process (e.g., gradient descent) is analogous to a system dissipating heat and settling into a thermal equilibrium at the bottom of a potential well.

1.2.3 The Loss Function as a Hamiltonian

Let $\theta \in \mathbb{R}^d$ denote the parameters of a model (e.g., weights of a neural network). Define a loss function $\mathcal{L}(\theta; \mathcal{D})$ that measures how poorly the model fits the training data. In the physicist's language, $\mathcal{L}$ plays the role of a *potential energy* or a *Hamiltonian*:

$$E(\theta) \equiv \mathcal{L}(\theta; \mathcal{D}). \tag{1.5}$$

The training process is then a search for the global (or at least a good local) minimum of this energy landscape. This perspective is powerful because it allows us to import intuition from statistical mechanics—e.g., the existence of many local minima (glassy behaviour), the role of temperature (learning rate), and the possibility of phase transitions as the model capacity changes.

1.2.4 Regularisation as a Prior and the Free Energy

In the Bayesian framework, we specify a prior $p(\theta)$ over parameters. The posterior is given by Bayes' theorem:

$$p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta)p(\theta)}{p(\mathcal{D})}. \tag{1.6}$$

Taking the negative logarithm yields

$$-\log p(\theta|\mathcal{D}) = -\log p(\mathcal{D}|\theta) - \log p(\theta) + \text{const.} \tag{1.7}$$

If we identify the negative log-likelihood with the empirical loss (e.g., mean squared error or cross-entropy), and the negative log-prior with a regularisation term, then minimising the posterior negative log-likelihood is exactly the same as minimising the regularised loss:

$$E(\theta) = \mathcal{L}(\theta) + \lambda R(\theta), \tag{1.8}$$

where λ acts like an inverse temperature or a coupling constant.

More formally, we can define a *variational free energy*

$$F = \mathbb{E}_{q(\theta)}[E(\theta)] - T\,S(q), \tag{1.9}$$

where $q(\theta)$ is a variational distribution over parameters, T is a temperature, and $S(q)$ is the entropy of the variational distribution. Minimising F balances the expected energy (data fit) against the entropy (model complexity). This is exactly the strategy used in variational inference (Chapter 6) and provides a principled way to avoid overfitting.

1.2.5 The Learning Dynamics as Approach to Equilibrium

Gradient descent—the workhorse of deep learning—can be seen as the deterministic evolution of a particle in the potential $E(\theta)$ under high friction (overdamped Langevin dynamics):

$$\dot{\theta} = -\nabla_\theta E(\theta), \tag{1.10}$$

where the dot denotes derivative with respect to a fictitious time (the iteration number). In the presence of stochasticity (minibatch noise), this becomes the Langevin equation

$$d\theta_t = -\nabla_\theta E(\theta_t)\, dt + \sqrt{2T}\, dW_t, \tag{1.11}$$

where W_t is a Wiener process and T is the effective temperature set by the learning rate and batch size. The stationary distribution of this process is the Gibbs distribution $p(\theta) \propto e^{-E(\theta)/T}$. Thus, training with stochastic gradient descent (SGD) is akin to simulating a thermalised system that eventually (under suitable conditions) samples from the posterior distribution. This connection has spurred a rich literature on the role of noise, the flatness of minima, and the generalisation properties of deep networks.

1.3 The Bias–Variance Tradeoff

One of the most fundamental concepts in ML is the bias–variance tradeoff. For a regression problem, suppose the true relationship is $y = f(x) + \epsilon$, with ϵ zero-mean noise of variance σ^2. We fit a model $\hat{f}(x; \mathcal{D})$ on a finite training set $\mathcal{D}$. The expected squared error at a test point x can be decomposed as

$$\mathbb{E}_{\mathcal{D}}[(y - \hat{f}(x))^2] = \underbrace{(f(x) - \mathbb{E}[\hat{f}(x)])^2}_{\text{bias}^2} + \underbrace{\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]}_{\text{variance}} + \sigma^2. \tag{1.12}$$

- **Bias** measures how much the average model deviates from the true function—it is the error due to overly simplistic assumptions (e.g., fitting a line to a quadratic curve).
- **Variance** measures how much the model fluctuates with different training sets—it is the error due to excessive sensitivity to the data.

1.3.1 Physical Analogy: Elastic Springs and Model Complexity

Imagine we are trying to approximate a complex potential $f(x)$ using a finite set of springs (basis functions). A simple model (few springs) has a large bias—it cannot bend enough to follow the true potential—but its shape is stable (low variance). A complex model (many springs) can bend to fit every training point, achieving low bias, but its shape is highly sensitive to the exact positions of the data points (high variance). This is directly analogous to the tradeoff between stiffness and flexibility in solid-state physics.

1.3.2 Regularisation and the Bias–Variance Tradeoff

Regularisation, such as L_2 weight decay, adds a penalty $\lambda\|\theta\|^2$ to the loss. In the bias–variance decomposition, this increases bias (because it pulls the model toward zero) but decreases variance (by shrinking the parameter space). The optimal λ balances these two effects, typically chosen via cross-validation. In physics, this is analogous to introducing a harmonic confining potential that stabilises an otherwise ill-posed inverse problem.

1.3.3 The Double Descent Phenomenon

Modern deep learning has revealed a more subtle behaviour: as model capacity grows beyond the point of interpolation, the test error *decreases* again—a phenomenon known as *double descent*. This challenges the classical U-shaped bias–variance curve and is an active area of research. From a physicist's perspective, this can be seen as entering a new regime where the effective theory (the network) is so overparameterised that it admits many equivalent solutions, and gradient descent implicitly selects the one with the best generalisation properties (the so-called "lazy training" or neural tangent kernel regime).

Practical: Measuring Entropy in Data

Before diving into algorithms, let's ground the concept of entropy with a simple computational experiment. The goal is to estimate the entropy of a continuous probability distribution from a finite sample—a task that appears in feature selection, mutual information estimation, and unsupervised learning.

Histogram estimator. The simplest method is to bin the data into K equal-width bins and compute the empirical probabilities $\hat{p}_k = n_k/N$, where n_k is the number of samples in bin k. The plug-in estimate is

$$\hat{H} = -\sum_{k=1}^{K} \hat{p}_k \log \hat{p}_k. \tag{1.13}$$

The number of bins K acts as a regularisation parameter: too few bins cause high bias (smoothing out structure); too many bins lead to high variance (many empty bins). This is precisely the bias–variance tradeoff in action.

Python snippet. Listing 1.1: Estimating entropy from a Gaussian sample.

```python
import numpy as np
import matplotlib.pyplot as plt

# Generate data from a 1D Gaussian
np.random.seed(42)
data = np.random.normal(0, 1, size=1000)

# Histogram entropy estimator
def entropy_histogram(data, bins=20):
    counts, _ = np.histogram(data, bins=bins, density=False)
    p = counts / np.sum(counts)
    p = p[p > 0]  # remove zeros
    return -np.sum(p * np.log(p))

# Compare for different bin numbers
bins_range = np.arange(5, 50, 2)
entropies = [entropy_histogram(data, b) for b in bins_range]

plt.plot(bins_range, entropies, 'o-')
plt.xlabel('Number of bins')
plt.ylabel('Estimated entropy (nats)')
plt.title('Entropy estimation vs. bin count')
plt.grid(True)
plt.show()
```

The true entropy of a standard normal is $\frac{1}{2}\log(2\pi e) \approx 1.419$ nats. The estimate will oscillate around this value; the optimal number of bins depends on the sample size and the smoothness of the underlying distribution. More sophisticated methods (kernel density estimation, nearest-neighbour estimators) exist, but the histogram approach already reveals the central theme of this book: every modelling choice involves a tradeoff between bias and variance, and physical intuition can guide us toward principled solutions.

Chapter Summary

We have established the conceptual pillars for the rest of the book:

1. Entropy and information provide a common language for uncertainty in both physics and learning.
2. The loss function is an energy landscape; training is a relaxation process.
3. Regularisation is a prior; the variational free energy balances fit and complexity.
4. The bias–variance tradeoff is a fundamental limitation, analogous to the flexibility–stability tradeoff in physical systems.

In the next chapter, we apply these ideas to the workhorse of supervised learning—linear and logistic regression—viewing them as harmonic oscillators in high-dimensional parameter spaces.

Chapter 2

Supervised Learning and Energy Landscapes

Supervised learning is the task of inferring a mapping from inputs $\mathbf{x} \in \mathbb{R}^d$ to outputs $y \in \mathbb{R}$ (regression) or $y \in \{1, \dots, C\}$ (classification) from a set of labelled examples $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$. In this chapter, we treat the parameters of our models as coordinates in a high-dimensional space, and the loss function as a potential energy landscape. Training becomes the motion of a particle under the influence of that potential, eventually settling into a minimum. This physical picture provides not only intuition but also direct mathematical tools for analysing convergence, generalisation, and robustness.

2.1 Linear and Logistic Regression

2.1.1 Linear Regression and the Quadratic Potential

The simplest supervised model assumes a linear relationship $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$. With the mean squared error (MSE) loss, the energy function is

$$E(\mathbf{w}, b) = \frac{1}{2N} \sum_{i=1}^{N} \Big(y_i - (\mathbf{w}^\top \mathbf{x}_i + b) \Big)^2 . \tag{2.1}$$

This is a quadratic function of the parameters—a paraboloid in $\mathbb{R}^{d+1}$—so it has a single global minimum that can be found analytically. The gradient with respect to $\mathbf{w}$ is

$$\nabla_{\mathbf{w}} E = -\frac{1}{N} \sum_{i=1}^{N} \mathbf{x}_i \Big(y_i - \mathbf{w}^\top \mathbf{x}_i - b \Big) . \tag{2.2}$$

Setting both gradients to zero yields the normal equations. In the augmented form with $\tilde{\mathbf{x}} = [1; \mathbf{x}]$ and $\boldsymbol{\theta} = [b; \mathbf{w}]$, the solution is

$$\boldsymbol{\theta}^* = \Big(\mathbf{X}^\top \mathbf{X} \Big)^{-1} \mathbf{X}^\top \mathbf{y}, \tag{2.3}$$

where $\mathbf{X}$ is the $N \times (d+1)$ design matrix. The matrix $\mathbf{X}^\top \mathbf{X}$ is proportional to the (uncentered) covariance of the inputs; its eigenvalues determine the curvature of the energy landscape. Flat directions (small eigenvalues) correspond to parameter combinations that barely affect the loss—a source of ill-conditioning that regularisation (Section 2.1.3) cures.

2.1.2 Logistic Regression and the Sigmoid Potential

For binary classification $y \in \{0, 1\}$, we model the probability

$$p(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}. \tag{2.4}$$

The cross-entropy loss (negative log-likelihood) gives the energy

$$E(\mathbf{w}, b) = -\frac{1}{N} \sum_{i=1}^{N} \Big[y_i \log \sigma(\mathbf{w}^\top \mathbf{x}_i + b) + (1 - y_i) \log\Big(1 - \sigma(\mathbf{w}^\top \mathbf{x}_i + b)\Big)\Big]. \tag{2.5}$$

Unlike MSE, this energy is convex but not quadratic—it saturates for well-classified points, meaning the gradient decays to zero far from the decision boundary. This is analogous to a potential that becomes flat at infinity, so the particle never feels a restoring force once it has pushed all examples to the correct side. If the data are linearly separable, the loss can be made arbitrarily close to zero as $\|\mathbf{w}\| \to \infty$, i.e., there is no finite minimum. Regularisation (next section) prevents this divergence by adding a confining term.

2.1.3 Regularisation as a Penalty Term

Adding a penalty to the energy function is equivalent to imposing a prior on the parameters.

Ridge (L_2) regularisation.

$$E_{\text{ridge}}(\boldsymbol{\theta}) = E(\boldsymbol{\theta}) + \lambda \|\boldsymbol{\theta}\|_2^2. \tag{2.6}$$

The quadratic penalty creates a harmonic oscillator potential around zero; the curvature of the Hessian increases by 2λ along every direction, improving conditioning. In Bayesian terms, this corresponds to a Gaussian prior $\boldsymbol{\theta} \sim \mathcal{N}(0, 1/(2\lambda))$.

Lasso (L_1) regularisation.

$$E_{\text{lasso}}(\boldsymbol{\theta}) = E(\boldsymbol{\theta}) + \lambda \|\boldsymbol{\theta}\|_1. \tag{2.7}$$

The absolute value penalty is an anharmonic potential that grows linearly with $|w_j|$. It produces sparse solutions (many $w_j = 0$) because the gradient is constant for positive/negative weights, pushing them exactly to zero when the data fit does not strongly favour them. This is analogous to a crystal field that forces certain degrees of freedom to freeze out.

2.2 Gradient Descent as Dissipative Dynamics

In the previous chapter, we introduced gradient descent as the deterministic motion of a particle in a viscous medium:

$$\dot{\boldsymbol{\theta}}(t) = -\nabla E(\boldsymbol{\theta}(t)). \tag{2.8}$$

Discretising time with step size η (the learning rate) gives the standard update:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \nabla E(\boldsymbol{\theta}_k). \tag{2.9}$$

For a quadratic potential $E(\boldsymbol{\theta}) = \frac{1}{2}\boldsymbol{\theta}^\top \mathbf{H} \boldsymbol{\theta}$, the exact solution is $\boldsymbol{\theta}_k = (\mathbf{I} - \eta \mathbf{H})^k \boldsymbol{\theta}_0$. Convergence requires $0 < \eta < 2/\lambda_{\max}(\mathbf{H})$. The slowest modes correspond to the smallest eigenvalues of $\mathbf{H}$—exactly the directions that are poorly constrained by the data.

2.2.1 Stochastic Gradient Descent as Langevin Dynamics

In practice, we compute the gradient on a random minibatch $\mathcal{B}$ of size $M \ll N$:

$$\widehat{\nabla} E = \frac{1}{M} \sum_{i \in \mathcal{B}} \nabla E_i(\boldsymbol{\theta}). \tag{2.10}$$

The minibatch gradient is the true gradient plus zero-mean noise whose covariance scales as $1/M$. Thus the update becomes

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \nabla E(\boldsymbol{\theta}_k) + \sqrt{2\eta T}\, \boldsymbol{\xi}_k, \tag{2.11}$$

where $\boldsymbol{\xi}_k \sim \mathcal{N}(0, \mathbf{I})$ and the effective temperature $T \propto \eta / M$. This is the Euler–Maruyama discretisation of the Langevin equation. The noise helps the particle escape sharp local minima and is believed to bias the system toward flat minima, which often generalise better—a phenomenon studied in the context of the "loss landscape" and "entropy of minima."

2.3 Support Vector Machines and Margin Maximization

The support vector machine (SVM) approaches classification from a geometric perspective. For a linearly separable binary dataset, we seek the hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ that maximises the margin—the distance from the hyperplane to the closest points (the support vectors). The margin is $2/\|\mathbf{w}\|$, so maximising it is equivalent to minimising $\frac{1}{2}\|\mathbf{w}\|^2$ subject to

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \quad \forall i. \tag{2.12}$$

This constrained optimisation can be recast as the unconstrained hinge loss:

$$E_{\text{SVM}}(\mathbf{w}, b) = \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^{N} \max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)). \tag{2.13}$$

The first term acts as an L_2 regulariser, while the hinge loss is zero for points that are correctly classified with margin at least 1, and grows linearly with the violation. The parameter C controls the tradeoff between margin width and training error—analogous to the inverse temperature in a soft-constraint system.

2.3.1 The Kernel Trick

The dual formulation of the SVM involves only inner products $\mathbf{x}_i^\top \mathbf{x}_j$. By replacing this with a kernel function $k(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$, we implicitly map the data into a high-dimensional feature space $\phi(\mathbf{x})$ without explicitly computing the coordinates. Common kernels include the radial basis function (RBF) $k(\mathbf{x}, \mathbf{x}') = \exp\bigl(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2\bigr)$, which corresponds to an infinite-dimensional feature space. This is analogous to choosing a complete set of basis functions in which the problem becomes linearly solvable—much like expanding a wavefunction in a basis of plane waves or spherical harmonics.

Practical: Implementing Linear Regression and SGD

Let us implement linear regression using both the closed-form solution and gradient descent. This will illustrate the concepts of energy landscapes, learning rates, and the effect of regularisation.

Data generation. We create a synthetic dataset $y = 2x_1 - 3x_2 + 0.5 + \epsilon$ with $\epsilon \sim \mathcal{N}(0, 0.1)$.

Listing 2.1: Linear regression with gradient descent from scratch.

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler

# Generate synthetic data
np.random.seed(42)
N = 1000
d = 2
X = np.random.randn(N, d)
true_w = np.array([2.0, -3.0])
true_b = 0.5
y = X @ true_w + true_b + 0.1 * np.random.randn(N)

# Add a column of ones for the bias
X_aug = np.hstack([np.ones((N, 1)), X])
theta_true = np.concatenate([[true_b], true_w])

# --- Closed-form solution ---
theta_cf = np.linalg.lstsq(X_aug, y, rcond=None)[0]
print(f"Closed-form theta: {theta_cf}")

# --- Gradient descent ---
def mse_gradient(X_aug, y, theta):
    N = len(y)
    return (1/N) * X_aug.T @ (X_aug @ theta - y)

def gradient_descent(X_aug, y, theta_init, lr=0.01, n_iter=500, reg
    =0.0):
    theta = theta_init.copy()
    history = [theta.copy()]
    for _ in range(n_iter):
        grad = mse_gradient(X_aug, y, theta) + reg * theta  # L2
            regularisation
        theta -= lr * grad
        history.append(theta.copy())
    return theta, np.array(history)

theta_init = np.random.randn(3)
theta_gd, history = gradient_descent(X_aug, y, theta_init, lr=0.05,
    n_iter=200, reg=0.01)

print(f"GD theta: {theta_gd}")

# Compare with sklearn
sk_lr = LinearRegression(fit_intercept=False)  # we already have bias
    in X_aug
sk_lr.fit(X_aug, y)
print(f"sklearn theta: {sk_lr.coef_}")

# Plot the convergence of the loss
losses = [np.mean((X_aug @ th - y)**2) / 2 for th in history]
plt.semilogy(losses)
plt.xlabel('Iteration')
plt.ylabel('MSE loss')
```

```
plt.title('Convergence of gradient descent')
plt.grid(True)
plt.show()
```

Discussion of the results. - The closed-form solution recovers the true parameters up to numerical precision. - Gradient descent approaches the same minimum, with convergence speed determined by the condition number of $\mathbf{X}^\top\mathbf{X}$. With L_2 regularisation (set 'reg¿0'), the Hessian is shifted by $\lambda\mathbf{I}$, improving conditioning and preventing the solution from exploding when features are correlated. - The learning rate η must be tuned: too high leads to divergence (the particle overshoots the potential well), too low results in slow convergence.

Stochastic version. Replace the full gradient with a minibatch gradient to observe the noisy (Langevin) dynamics. The noise can be seen as thermal fluctuations that help escape shallow local minima—though for this convex problem, the benefit is primarily computational speed.

Listing 2.2: Stochastic gradient descent for linear regression.

```
def sgd(X_aug, y, theta_init, lr=0.01, epochs=100, batch_size=32, reg
    =0.0):
    theta = theta_init.copy()
    N = len(y)
    for epoch in range(epochs):
        indices = np.random.permutation(N)
        for start in range(0, N, batch_size):
            batch_idx = indices[start:start+batch_size]
            X_b = X_aug[batch_idx]
            y_b = y[batch_idx]
            grad = (1/len(batch_idx)) * X_b.T @ (X_b @ theta - y_b) +
                reg * theta
            theta -= lr * grad
    return theta

theta_sgd = sgd(X_aug, y, theta_init, lr=0.01, epochs=100)
print(f"SGD theta: {theta_sgd}")
```

The SGD estimate will fluctuate around the optimum; the variance of these fluctuations is proportional to η and inversely proportional to the batch size, exactly as predicted by the Langevin description.

Chapter Summary

We have seen how supervised learning can be understood as the minimisation of an energy landscape:

- Linear regression gives a quadratic (harmonic) potential with a single minimum; logistic regression yields a convex but saturating potential.
- Regularisation adds confining terms—L_2 is a harmonic trap, L_1 is an anharmonic one that induces sparsity.
- Gradient descent is the discrete-time version of overdamped motion in this potential; stochasticity from minibatches provides thermal noise that aids exploration and generalisation.
- The SVM maximises the margin, effectively placing a repulsive "buffer" around the decision boundary, with kernels allowing for nonlinear feature spaces.

In Chapter 3, we move to unsupervised learning, where the energy landscape is often defined without labels—clustering and dimensionality reduction become problems of finding low-energy configurations of the data itself.

Chapter 12

Conclusion

We have come a long way. From the foundations of entropy and energy landscapes, through supervised and unsupervised learning, deep neural networks, generative models, probabilistic inference, reinforcement learning, quantum machine learning, case studies, and the challenges that lie ahead, we have traversed the modern landscape of machine learning through the eyes of a physicist. This concluding chapter reflects on the journey, synthesises the key themes, and offers some final thoughts on the road ahead.

The Journey in Retrospect

Our journey began with a simple but profound observation: machine learning and physics share a common language. The concepts of entropy, free energy, symmetry, and dynamics that we use to describe physical systems also provide a natural framework for understanding learning algorithms.

In Chapter 1, we established the conceptual bridge: Shannon entropy and Gibbs entropy are two sides of the same coin; the loss function is an energy landscape; gradient descent is dissipative dynamics in a potential. We learned that the bias-variance tradeoff—the fundamental tension between model complexity and generalisation—is a manifestation of the same tradeoff between flexibility and stability that physicists encounter when choosing models.

In Chapters 2 and 3, we explored supervised and unsupervised learning through this lens. Linear and logistic regression became harmonic and saturating potentials; PCA became the diagonalisation of a covariance matrix; K-means became a first-order phase transition. We saw that regularisation is the imposition of a prior—a confining potential that prevents overfitting—and that the kernel trick is the choice of a basis in which the problem becomes linearly separable.

Chapters 4 and 5 took us into the heart of deep learning. We saw that neural networks are spin systems with learned couplings: the perceptron is an Ising spin in an external field; Hopfield networks are spin glasses with Hebbian couplings; Boltzmann machines sample from thermal distributions. Backpropagation became adjoint sensitivity analysis in discrete time, and convolutional neural networks emerged as a natural consequence of translational invariance—the deep learning analogue of short-range interactions in lattice systems. Attention mechanisms revealed themselves as all-to-all pairwise interactions with a Boltzmann factor, and the Transformer architecture became a manifestation of long-range correlations in physical systems.

Chapter 6 introduced generative models, where we learned to sample from the data distribution itself. Variational autoencoders minimised the variational free energy, and diffusion models reversed a stochastic process governed by Langevin dynamics. We saw how the score of the data distribution acts as a drift, guiding particles from noise to structure—a beautiful demonstration of the connection between generative modelling and non-equilibrium thermodynamics.

In Chapter 7, we deepened the probabilistic perspective, exploring Bayesian inference, Gaussian processes, and variational inference. We learned how to quantify uncertainty, incorporate

prior knowledge, and compare models—essential skills for any physicist who wants to make reliable predictions from data. We saw that Bayesian inference is structurally identical to the Boltzmann distribution, and that the marginal likelihood is the partition function of a physical system.

Chapter 8 connected reinforcement learning to optimal control and classical mechanics. The Bellman equation became the discrete Hamilton-Jacobi-Bellman equation, and exploration-exploitation emerged as a temperature-controlled tradeoff, analogous to simulated annealing. We saw that deep Q-networks with experience replay and target networks stabilise learning, much as symplectic integrators preserve the structure of Hamiltonian dynamics.

Chapter 9 took us into the quantum realm, where the exponential growth of the Hilbert space offers new possibilities for representation and computation. We learned about parameterised quantum circuits, quantum kernels, and variational quantum classifiers—the building blocks of a new paradigm that may one day surpass classical methods. The parameter shift rule emerged as the quantum analogue of backpropagation, and barren plateaus reminded us of the vanishing gradient problem in classical deep networks.

Chapter 10 grounded these ideas in real physics applications: event classification at the LHC, gravitational wave detection, phase transition identification, and quantum state tomography. These case studies demonstrated that ML is not just a theoretical curiosity but a practical tool that is transforming the physical sciences—from the smallest scales of quantum systems to the largest scales of the cosmos.

Finally, Chapter 11 surveyed the challenges and future directions: interpretability, data scarcity, physics-informed learning, and hardware limitations. We saw that the path forward lies not in replacing physical intuition with black-box algorithms, but in combining the two to create models that are both powerful and transparent. Physics-informed neural networks, equivariant architectures, and foundation models for physics represent the vanguard of this synthesis.

Key Lessons Learned

Throughout this book, several recurring themes have emerged:

1. Physics Provides a Natural Language for ML

The conceptual toolkit of physics—statistical mechanics, dynamical systems, field theory, quantum mechanics—maps beautifully onto the concepts of machine learning. This is not a coincidence: both fields deal with complex systems, emergent behaviour, and the extraction of structure from data. By learning to speak the language of ML in physical terms, we can understand algorithms more deeply and develop new ones more creatively.

2. Uncertainty Matters

Physicists are accustomed to error bars, confidence intervals, and statistical significance. ML models that only produce point estimates are incomplete. Bayesian methods, Gaussian processes, and variational inference provide the tools to quantify uncertainty, which is essential for trustworthy scientific discovery.

3. Symmetry Is a Powerful Inductive Bias

The symmetries of a physical system constrain its behaviour. By building these symmetries into ML models—through CNNs (translation invariance), equivariant networks (rotation invariance), or Hamiltonian neural networks (energy conservation)—we can dramatically improve sample efficiency and generalisation.

4. Models Are Not Reality

Every model is an approximation. A neural network that achieves 99% accuracy on a test set is still a simplified representation of the underlying process. As physicists, we must remain critical: we must validate our models, understand their limitations, and never confuse correlation with causation.

5. The Best Models Combine Physics and Data

Pure data-driven approaches can be powerful, but they often require vast amounts of data and lack interpretability. Physics-informed approaches that incorporate known laws, symmetries, and conservation principles can achieve more with less data and provide insights into the underlying mechanisms.

6. The Future Is Interdisciplinary

The most exciting advances will come at the intersections: ML with quantum computing, ML with experimental physics, ML with materials science, ML with cosmology. Physicists who can speak both languages—who understand the algorithms and the physics—will be in a unique position to drive these advances.

A Call to Action

If this book has achieved its purpose, you should now have a solid foundation in the core concepts and practical tools of machine learning, along with an appreciation for the deep connections to physics. But reading a book is only the beginning. The real learning happens when you apply these ideas to your own problems.

Here is a roadmap for your next steps:

1. **Start small.** Take a simple physics problem you already understand and apply an ML method to it. Compare the ML solution to the analytical solution. This builds intuition and trust.

2. **Get your hands dirty.** Write code. Run experiments. Visualise results. The code examples in this book are starting points; modify them, break them, and see what happens.

3. **Join the community.** Attend conferences and workshops, read the arXiv, participate in online forums. The ML community is remarkably open and welcoming.

4. **Collaborate.** Partner with computer scientists, statisticians, or other physicists who have complementary skills. The best science is done in teams.

5. **Stay curious.** The field is evolving rapidly. New algorithms, architectures, and applications emerge every week. Keep learning, keep questioning, and keep pushing the boundaries.

Final Reflections

As we close this book, we return to the fundamental question that motivates both physics and machine learning: *How can we make sense of the world?* Physics has always sought to describe nature with mathematical laws, to extract simplicity from complexity, to predict the future from the present. Machine learning is the latest chapter in this grand enterprise—a set of tools that allows us to find patterns that our equations may have missed, to discover new phenomena that

our theories did not anticipate, and to build models that are more flexible and adaptive than we could have imagined.

But machine learning is not a replacement for physics. It is an augmentation. The great discoveries of the future will come not from algorithms alone, but from the synergy between human intuition and computational power. As a physicist, you are uniquely equipped to guide this synergy: you understand the importance of symmetries, conservation laws, and physical principles; you know how to ask the right questions; you are trained to be sceptical and rigorous.

The journey of a thousand miles begins with a single step. You have taken that step by reading this book. Now go forth and explore. The universe is full of data, and there is much yet to discover.

"The important thing is not to stop questioning. Curiosity has its own reason for existing."
— Albert Einstein

Acknowledgements

This book would not exist without the pioneering work of countless physicists, computer scientists, and mathematicians who have built the foundations of machine learning. We are particularly indebted to the authors of the review article that inspired this book—Pankaj Mehta, Marin Bukov, Ching-Hao Wang, Alexandre G.R. Day, Clint Richardson, Charles K. Fisher, and David J. Schwab—whose work demonstrated that a physicist's perspective on machine learning is both valuable and illuminating.

We also thank the open-source community for creating the tools—NumPy, scikit-learn, PyTorch, Qiskit, and many others—that make machine learning accessible to all. And we thank the countless researchers who share their code, data, and insights, advancing the field for the benefit of everyone.

Finally, we thank you, the reader, for joining us on this journey. We hope this book has been a useful companion, and we wish you all the best in your future explorations at the intersection of physics and machine learning.

Appendix A

Mathematical, Physical, and Computational Toolkit

This appendix collects the essential mathematical and physical concepts used throughout the book. It is not intended to be a comprehensive textbook, but rather a quick reference and refresher. For deeper treatments, we refer the reader to the standard texts in the bibliography.

A.1 Linear Algebra

Linear algebra is the language of machine learning. Data are vectors, models are matrices, and learning is the optimisation of linear transformations.

A.1.1 Vectors and Matrices

A vector $\mathbf{x} \in \mathbb{R}^d$ is an ordered list of d real numbers. The inner product (dot product) of two vectors is

$$\mathbf{x}^\top \mathbf{y} = \sum_{i=1}^{d} x_i y_i. \tag{A.1}$$

A matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ is a rectangular array of numbers. Matrix multiplication is defined as

$$(\mathbf{AB})_{ij} = \sum_{k} A_{ik} B_{kj}. \tag{A.2}$$

The transpose $\mathbf{A}^\top$ swaps rows and columns: $(\mathbf{A}^\top)_{ij} = A_{ji}$.

A.1.2 Eigenvalues and Eigenvectors

A scalar λ and non-zero vector $\mathbf{v}$ are an eigenvalue and eigenvector of a square matrix $\mathbf{A}$ if

$$\mathbf{A}\mathbf{v} = \lambda \mathbf{v}. \tag{A.3}$$

For symmetric matrices, the eigenvalues are real and the eigenvectors form an orthonormal basis. This decomposition is fundamental to PCA (Chapter 3), where the covariance matrix is diagonalised.

A.1.3 Singular Value Decomposition (SVD)

Every matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ can be factorised as

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top, \tag{A.4}$$

where $\mathbf{U}$ and $\mathbf{V}$ are orthogonal matrices, and $\mathbf{\Sigma}$ is diagonal with non-negative singular values σ_i. The SVD is the workhorse of PCA and low-rank approximations.

A.1.4 Matrix Derivatives

For vectors and matrices, we often need derivatives of scalar functions. Useful identities include:

$$\frac{\partial}{\partial \mathbf{x}} \mathbf{a}^\top \mathbf{x} = \mathbf{a}, \tag{A.5}$$

$$\frac{\partial}{\partial \mathbf{x}} \mathbf{x}^\top \mathbf{A} \mathbf{x} = (\mathbf{A} + \mathbf{A}^\top)\mathbf{x}, \tag{A.6}$$

$$\frac{\partial}{\partial \mathbf{A}} \operatorname{tr}(\mathbf{B}\mathbf{A}) = \mathbf{B}^\top. \tag{A.7}$$

These identities are essential for deriving gradient updates in Chapters 2 and 5.

A.2 Calculus and Optimisation

Optimisation is at the heart of machine learning—we adjust parameters to minimise a loss function.

A.2.1 Gradients and Hessians

For a scalar function $f(\mathbf{x})$, the gradient is the vector of first partial derivatives:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \partial f / \partial x_1 \\ \vdots \\ \partial f / \partial x_d \end{bmatrix}. \tag{A.8}$$

The Hessian $\mathbf{H}$ is the matrix of second derivatives:

$$H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}. \tag{A.9}$$

The Hessian determines the curvature of the loss landscape and is used in Newton's method and convergence analysis.

A.2.2 The Chain Rule

For composite functions, the chain rule is

$$\frac{d}{dt} f(\mathbf{x}(t)) = \nabla f(\mathbf{x}(t))^\top \frac{d\mathbf{x}}{dt}. \tag{A.10}$$

This is the foundation of backpropagation (Chapter 5), where the gradient of the loss with respect to the weights is computed by propagating derivatives backward through the network.

A.2.3 Gradient Descent

Gradient descent updates parameters to minimise a differentiable loss $L(\boldsymbol{\theta})$:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \nabla L(\boldsymbol{\theta}_k), \tag{A.11}$$

where η is the learning rate. Convergence to a local minimum is guaranteed for sufficiently small η if L is smooth. In practice, we use stochastic gradient descent (SGD) with minibatches (Chapter 2), where the gradient is replaced by a noisy estimate.

A.2.4 Lagrange Multipliers

For constrained optimisation, we introduce Lagrange multipliers. To minimise $f(\mathbf{x})$ subject to $g(\mathbf{x}) = 0$, we solve

$$\nabla f = \lambda \nabla g. \tag{A.12}$$

This technique is used in the dual formulation of support vector machines (Chapter 2) and in maximum entropy methods.

A.3 Probability and Information Theory

Machine learning is fundamentally about reasoning under uncertainty. Probability theory provides the language.

A.3.1 Key Distributions

The following distributions appear frequently:

- **Bernoulli:** $p(x) = p^x (1-p)^{1-x}$ for $x \in \{0, 1\}$. Used in binary classification.
- **Multinoulli (Categorical):** Generalisation to K categories.
- **Gaussian (Normal):** $p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$. The workhorse for regression and latent variable models.
- **Poisson:** $p(k) = \frac{\lambda^k e^{-\lambda}}{k!}$ for counting processes.
- **Beta:** $p(x) \propto x^{\alpha-1}(1-x)^{\beta-1}$. Conjugate prior for Bernoulli.
- **Gamma:** Conjugate prior for the precision of a Gaussian.

A.3.2 Expectation, Variance, and Moments

The expectation of a random variable X is

$$\mathbb{E}[X] = \int x \, p(x) \, dx. \tag{A.13}$$

The variance is $\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$. Higher moments are used in the bias–variance decomposition (Chapter 1).

A.3.3 Entropy and KL Divergence

The Shannon entropy of a distribution is

$$H(p) = -\mathbb{E}_p[\log p(x)]. \tag{A.14}$$

The Kullback–Leibler (KL) divergence between p and q is

$$D_{\mathrm{KL}}(p \,\|\, q) = \mathbb{E}_p\left[\log \frac{p(x)}{q(x)}\right]. \tag{A.15}$$

The KL divergence is a measure of how much information is lost when using q to approximate p. It underpins the ELBO in VAEs (Chapter 6) and the objective of logistic regression (Chapter 2).

A.3.4 Bayes' Theorem

Bayes' theorem updates beliefs given data:

$$p(\boldsymbol{\theta}|\mathcal{D}) = \frac{p(\mathcal{D}|\boldsymbol{\theta})\, p(\boldsymbol{\theta})}{p(\mathcal{D})}. \tag{A.16}$$

This is the foundation of Bayesian inference (Chapter 10), where we treat parameters as random variables and incorporate prior knowledge.

A.4 Statistical Mechanics Essentials

Statistical mechanics provides the physical intuition for many ML concepts. The connections are not superficial—they are mathematically rigorous.

A.4.1 The Boltzmann Distribution

The probability of a system being in a state with energy E_i at temperature T is

$$p_i = \frac{1}{Z} e^{-E_i/(k_B T)}, \tag{A.17}$$

where the partition function is $Z = \sum_i e^{-E_i/(k_B T)}$. This distribution minimises the free energy $F = \langle E \rangle - TS$. In ML, the same structure appears in energy-based models, RBMs (Chapter 4), and attention mechanisms (Chapter 5).

A.4.2 Free Energy and the Variational Principle

The variational free energy is

$$F = \mathbb{E}_q[E] - T\, S(q), \tag{A.18}$$

where q is a trial distribution. Minimising F with respect to q yields the Boltzmann distribution. This is exactly the principle used in variational inference and VAEs (Chapter 6).

A.4.3 The Ising Model

The Ising model consists of binary spins $s_i \in \{-1, +1\}$ with Hamiltonian

$$H = -\sum_{i<j} J_{ij} s_i s_j - \sum_i h_i s_i. \tag{A.19}$$

This is the Hopfield network Hamiltonian (Chapter 4). The phase transition of the Ising model—from ordered to disordered—mirrors the capacity and spurious states of associative memories.

A.4.4 Langevin Dynamics and Stochastic Processes

The overdamped Langevin equation describes a particle in a potential with thermal noise:

$$d\mathbf{x} = -\nabla E(\mathbf{x})\, dt + \sqrt{2T}\, d\mathbf{W}. \tag{A.20}$$

This is the continuous-time version of stochastic gradient descent (Chapter 2) and the reverse process in diffusion models (Chapter 6).

A.5 Python and ML Libraries Quick Start

All code examples in this book use Python and standard ML libraries. This section provides a minimal setup guide.

A.5.1 Installation

We recommend using Anaconda or Miniconda to manage packages. Install the required libraries with:

```
conda create -n ml_physics python=3.10
conda activate ml_physics
conda install numpy matplotlib scikit-learn jupyter
conda install pytorch torchvision torchaudio -c pytorch
pip install qiskit qiskit-aer lime gym
```

A.5.2 NumPy: Basic Tensor Operations

NumPy provides n-dimensional arrays and linear algebra routines:

Listing A.1: NumPy basics.

```python
import numpy as np

# Create vectors and matrices
x = np.array([1.0, 2.0, 3.0])
A = np.array([[1, 2], [3, 4]])

# Operations
y = x @ A           # matrix-vector product
u, s, vt = np.linalg.svd(A)  # SVD
eigvals, eigvecs = np.linalg.eig(A @ A.T)
```

A.5.3 scikit-learn: Standard ML Algorithms

scikit-learn provides high-level implementations of many algorithms:

Listing A.2: scikit-learn examples.

```python
from sklearn.linear_model import LinearRegression
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.gaussian_process import GaussianProcessRegressor

# Fit a linear model
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

A.5.4 PyTorch: Deep Learning and Automatic Differentiation

PyTorch is the primary library for deep learning in this book:

Listing A.3: PyTorch basics.

```python
import torch
import torch.nn as nn
import torch.optim as optim
```

```

# Tensors and gradients
x = torch.tensor([1.0, 2.0], requires_grad=True)
y = (x ** 2).sum()
y.backward()   # computes gradients

# Define a simple network
model = nn.Sequential(
    nn.Linear(10, 20),
    nn.ReLU(),
    nn.Linear(20, 1)
)
optimizer = optim.Adam(model.parameters(), lr=1e-3)
```

A.5.5 Qiskit: Quantum Circuits

Qiskit is used for quantum machine learning examples:

Listing A.4: Qiskit basics.

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator

# Create a Bell state
qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)
qc.measure_all()

# Simulate
sim = AerSimulator()
job = sim.run(qc, shots=1024)
counts = job.result().get_counts()
```

Further Reading

For deeper coverage of the mathematical foundations, we recommend:

- Strang, G. (2016). *Introduction to Linear Algebra*. Wellesley-Cambridge Press.
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. (Chapters 1–2 for probability and linear algebra)
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. (Chapter 2 for linear algebra, Chapter 3 for probability)
- Kardar, M. (2007). *Statistical Physics of Particles*. Cambridge University Press. (For the statistical mechanics background)

Glossary

Activation function
: A non-linear function applied to the output of a neuron, e.g., ReLU, sigmoid, or tanh. It introduces non-linearity, enabling the network to approximate complex functions.

Adjoint method
: A technique for computing gradients of a loss with respect to parameters by solving a backward-time equation; backpropagation is the discrete analogue.

Attention
: A mechanism that computes a weighted sum of values based on query–key similarity; it allows models to dynamically focus on relevant parts of the input.

Autoencoder
: A neural network trained to reconstruct its input through a low-dimensional bottleneck; used for dimensionality reduction and generative modelling.

Bayesian inference
: A statistical framework where parameters are treated as random variables with a prior distribution, updated to a posterior given observed data.

Bias–variance tradeoff
: The fundamental tradeoff between the error due to overly simplistic assumptions (bias) and error due to sensitivity to training data (variance).

Boltzmann machine
: A stochastic neural network with binary units, governed by the Boltzmann distribution; used for generative modelling and feature learning.

Convolution
: A linear operation that applies a learned kernel across spatial (or temporal) positions, enforcing translation equivariance.

Deep Q-Network (DQN)
: A reinforcement learning algorithm that uses a deep neural network to approximate the action-value function, with experience replay and a target network.

Diffusion model
: A generative model that gradually adds noise to data (forward process) and learns to reverse this process, generating samples from pure noise.

ELBO (Evidence Lower Bound)
: A lower bound on the log-likelihood; maximising it approximates posterior inference and is the objective for VAEs.

Energy landscape
: The loss function viewed as a potential energy surface in parameter space; training corresponds to moving downhill to a minimum.

Entropy
: A measure of uncertainty or information content; Shannon entropy for random variables, Gibbs entropy in statistical mechanics.

Gaussian process
: A non-parametric Bayesian model that defines a prior over functions, providing predictive means and uncertainties.

Gradient descent
: An optimisation algorithm that iteratively updates parameters in the direction of steepest descent of the loss function.

Hopfield network
: A recurrent neural network that functions as associative memory; its dynamics minimise a spin-glass Hamiltonian.

Ising model
: A lattice model of binary spins with nearest-neighbour interactions; foundational to statistical mechanics and neural network analogies.

Kernel trick
: A method to implicitly map data into a high-dimensional feature space by replacing inner products with a kernel function, enabling non-linear classifiers.

Langevin dynamics
: A stochastic differential equation describing a particle in a potential with thermal noise; underpins SGD and diffusion models.

Markov Decision Process (MDP)
: A mathematical framework for sequential decision-making with states, actions, transitions, rewards, and a discount factor.

Parameterised quantum circuit (PQC)
: A quantum circuit with tunable gates, analogous to a classical neural network; trained via variational methods.

Partition function
: A normalising constant in statistical mechanics (and Bayesian inference) that sums over all states; central to energy-based models.

Physics-informed neural network (PINN)
: A neural network trained to satisfy a differential equation by including the PDE residual in the loss function.

Reinforcement learning
: A learning paradigm where an agent learns to maximise cumulative reward by interacting with an environment.

Restricted Boltzmann Machine (RBM)
: A bipartite Boltzmann machine with no intra-layer connections, enabling efficient training via contrastive divergence.

Singular Value Decomposition (SVD)
: A matrix factorisation that decomposes any matrix into orthogonal and diagonal components; used in PCA and low-rank approximations.

Stochastic gradient descent (SGD)
: Gradient descent using a random subset (minibatch) of training data; introduces noise that can help escape local minima.

Support vector machine (SVM)
: A classifier that finds the hyperplane maximising the margin between classes, often using the kernel trick.

Transformer
: An architecture based on self-attention mechanisms, processing sequences with all-to-all interactions; the foundation of modern large language models.

Variational autoencoder (VAE)
: A generative model that learns a latent space by maximising the ELBO; uses the reparameterisation trick for differentiable sampling.

Bibliography

[1] Mehta, P., Bukov, M., Wang, C.-H., Day, A. G. R., Richardson, C., Fisher, C. K., & Schwab, D. J. (2019). *A high-bias, low-variance introduction to Machine Learning for physicists.* Physics Reports, 810, 1-124.

[2] Hopfield, J. J. (1982). *Neural networks and physical systems with emergent collective computational abilities.* Proceedings of the National Academy of Sciences, 79(8), 2554-2558.

[3] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning.* MIT Press.

[4] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning.* Springer.

[5] Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective.* MIT Press.

[6] Carleo, G., Cirac, I., Cranmer, K., Daudet, L., Schuld, M., Tishby, N., Vogt-Maranto, L., & Zdeborová, L. (2019). *Machine learning and the physical sciences.* Reviews of Modern Physics, 91(4), 045002.

[7] Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations.* Journal of Computational Physics, 378, 686-707.

[8] Karniadakis, G. E., Kevrekidis, I. G., Lu, L., Perdikaris, P., Wang, S., & Yang, L. (2021). *Physics-informed machine learning.* Nature Reviews Physics, 3(6), 422-440.